

# CSCI 3334 — Systems Programming

Fall 2026 · University of Texas Rio Grande Valley

## Course Information

---

|                     |                                                                                |
|---------------------|--------------------------------------------------------------------------------|
| <b>Course</b>       | CSCI 3334 — Systems Programming, Section 03                                    |
| <b>Term</b>         | Fall 2026 · August 26 – December 10                                            |
| <b>Credit hours</b> | 3                                                                              |
| <b>Modality</b>     | Traditional, face-to-face                                                      |
| <b>Meets</b>        | Monday & Wednesday, 3:30 – 4:45 PM                                             |
| <b>Location</b>     | Edinburg Campus · EIEAB (Interdisciplinary Engineering & Academic), Room 2.205 |

---

## Instructor Information

---

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <b>Instructor</b>   | Dr. Pu Jiao                                           |
| <b>Email</b>        | pu.jiao@utrgv.edu                                     |
| <b>Office</b>       | EIEAB 3.237                                           |
| <b>Office hours</b> | 2:20–3:20 PM, Monday and Wednesday, or by appointment |

---

## Communication

Use your **UTRGV email account** for all course correspondence — university policy requires electronic business to go through official university systems. Put **CSCI 3334** in the subject line with a short description of what you need.

I answer email during working hours on weekdays. If the question is about code that is misbehaving, office hours will get you a better answer in five minutes than email will in a day. Bring the code.

## Course Description

This course covers the design and implementation of system software. It investigates the relationship between software design and machine architecture. Topics may include assemblers, macro-processors, compilers, loaders, debugging environments, program development and archival tools, command language interpreters (shells), file systems, I/O support, processes, threads, and inter-process communication.

**Prerequisites:** Grade of C or better in [CSCI 2380 or CMPE 2380] **and** [CSCI 2333 or CMPE 2333 or ELEE 3435 or CMPE 3437 or EECE 3435].

## What the course is actually about

High-level languages hide the machine from you. This course pulls back the curtain. By the end you will understand:

- why a program dies with “segmentation fault”
- how `malloc` and `free` actually work
- what happens between pressing Enter and your shell running a command
- why two threads updating one variable can produce a wrong answer

- how CPU caches make some programs 10× faster with no change to the algorithm

These are the foundations for operating systems, compilers, databases, networking, security, embedded systems and high-performance computing. CSCI 3334 is the prerequisite for **CSCI 4334 Operating Systems** and **CSCI 4327 Compiler Construction**.

## Course Learning Objectives

A student who passes this course should be able to:

1. Explain how C data types are represented in memory, and predict the results of integer and floating-point operations including overflow and rounding.
2. Read x86-64 assembly produced by a compiler and relate it back to the C it came from.
3. Explain how the kernel isolates processes, and use system calls for process creation, file I/O and signalling.
4. Explain virtual memory and dynamic allocation, and reason about what a memory access costs.
5. Write correct concurrent C using threads, mutexes and condition variables.
6. Relate all of the above to real systems, including the machine-level costs that dominate modern AI and machine-learning workloads.

## Required Materials

**Textbook:** Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd Edition. ISBN-13 978-0134092669. Companion site: [csapp.cs.cmu.edu](http://csapp.cs.cmu.edu).

Lecture notes, slides and lab handouts are posted on the course site at no cost.

## Language and tools

All programming is in **C (C17)**. We use C because it is the language of systems — the Linux kernel, database engines, web servers and language runtimes are written in it.

| Tool     | Use                                        |
|----------|--------------------------------------------|
| GCC      | <code>gcc -std=c17 -Wall -Wextra -g</code> |
| GDB      | debugger                                   |
| Make     | build system                               |
| Valgrind | memory error and leak detection            |
| Git      | version control and submission             |

## What you should already be comfortable with

- basic C syntax: variables, loops, functions, pointers, structs
- dynamic memory: `malloc` and `free`
- working at a terminal with an editor and a compiler; no IDE required

### Note

If you have not written much C, the first two weeks are built to bring you up to speed, and **Lab 0 is a self-check worth up to 3.0 bonus percentage points** toward your final grade — it's pure upside, with no penalty for anything you can't complete. It exists to tell you — and me — where you stand while there is still time to act on it. Take it seriously, and come to office hours if it goes badly. That is exactly what it is for.

## Technology Needs

A working laptop and a reliable place to work. A tablet is not sufficient for this course. The open CS labs on campus are a fine alternative to your own machine, if less convenient.

All coursework runs on Linux. If you do not have a Linux machine, WSL2 on Windows or a virtual machine is fine — we get everyone set up in week 1.

## Tentative Calendar of Activities

*Tentative and subject to change. Changes are announced in class and posted on the course site.*

| Week   | Dates          | Topics                                                         | Labs                                                           |
|--------|----------------|----------------------------------------------------------------|----------------------------------------------------------------|
| 1      | Aug 26–28      | Introduction & C Refresher                                     | Lab 0 out (bonus warmup)                                       |
| 2      | Aug 31 – Sep 4 | Data Representation I · Data Representation II                 |                                                                |
| 3      | Sep 7–11       | Data Representation III<br>( <i>Labor Day — no class</i> )     | Lab 0 due · Lab 1 out                                          |
| 4      | Sep 14–18      | Data Representation IV · Floating Point                        |                                                                |
| 5      | Sep 21–25      | Assembly I · Assembly II                                       | Lab 1 due · Lab 2 out                                          |
| 6      | Sep 28 – Oct 2 | Assembly III · Kernel I                                        |                                                                |
| 7      | Oct 5–9        | Kernel II · Kernel III                                         | Lab 2 due                                                      |
| 8      | Oct 12–16      | Kernel IV · <b>Midterm exam</b>                                |                                                                |
| 9      | Oct 19–23      | Storage I · Storage II                                         |                                                                |
| 10     | Oct 26–30      | Memory Hierarchy I · Memory Hierarchy II                       |                                                                |
| 11     | Nov 2–6        | Memory Hierarchy III · Process Control I                       | Lab 3 out (Cache Simulator) · Lab 4 out (Simple Shell)         |
| 12     | Nov 9–13       | Process Control II · Process Control III                       |                                                                |
| 13     | Nov 16–20      | Concurrency I · Concurrency II                                 | Lab 3 due · Lab 4 Level 1 checkpoint · Lab 5 out (Thread Pool) |
| 14     | Nov 23–27      | Concurrency III · Systems in the Real World                    | Lab 5 Level 1 checkpoint (in class)                            |
| 15     | Nov 30 – Dec 4 | Concurrency IV · Course Review ( <i>Study Day — no class</i> ) | Lab 4 due · Lab 5 due                                          |
| Finals | Dec 4–10       | <b>Final exam</b>                                              |                                                                |

## Assessment of Learning

### Assignments

| Lab   | Topic                                        | Out    | Due    | Weight                              |
|-------|----------------------------------------------|--------|--------|-------------------------------------|
| Lab 0 | Warmup — environment, GDB, Valgrind          | Aug 26 | Sep 9  | bonus (up to 3.0 percentage points) |
| Lab 1 | Data representation — bit manipulation       | Sep 9  | Sep 23 | 10%                                 |
| Lab 2 | Assembly — reverse engineering and the bomb  | Sep 23 | Oct 7  | 10%                                 |
| Lab 3 | Cache simulator                              | Nov 2  | Nov 16 | 12%                                 |
| Lab 4 | Simple shell — fork/exec, pipes, redirection | Nov 4  | Nov 30 | 14%                                 |
| Lab 5 | Thread pool                                  | Nov 18 | Dec 2  | 14%                                 |

Every lab is graded in **levels**, and level 1 of each lab is worth enough on its own to pass that lab. The later levels separate a B from an A; they are not what decides whether you pass. Each lab ships with the same test harness used to grade it, so you always know where you stand before submitting.

Labs 2 through 5 have a **checkpoint** one week in: show me level 1 working. Each completed checkpoint is recorded as participation and earns **+0.5 bonus percentage points**, up to the course bonus limit. Its purpose is to catch you while there is still time, not to grade you twice.

## Grading

| Component                                       | Weight                                                               |
|-------------------------------------------------|----------------------------------------------------------------------|
| Labs (5 graded)                                 | 60%                                                                  |
| Midterm exam — Wednesday, October 14            | 15%                                                                  |
| Final exam — registrar-assigned slot (Dec 4–10) | 20%                                                                  |
| Attendance                                      | 5%                                                                   |
| Participation and checkpoints (bonus)           | +0.5 points per recorded participation/checkpoint, up to +5.0 points |
| Lab 0 (bonus)                                   | +3.0 percentage points                                               |

Bonus points are added to the final course average, capped at 100%.

| Letter | Numerical |
|--------|-----------|
| A      | 90–100    |
| B      | 80–89     |
| C      | 70–79     |
| D      | 60–69     |
| F      | below 60  |

A grade of **C or better** is required for this course to satisfy the prerequisite for CSCI 4334 and CSCI 4327.

## Late work

10% penalty per day, to a maximum of three days, after which the lab is not accepted. Extensions for documented circumstances are granted — ask **before** the deadline wherever possible.

## Attendance

Attendance is expected. This is a face-to-face course, and the in-class work — live debugging, worked examples, quizzes — is not something you can reconstruct from the slides afterwards.

Absences are officially excused for: participation in university-sponsored activities; accommodations documented through Student Accessibility Services; observance of religious holy days; mandatory military service; and severe illness requiring a physician's attention, or the death of an immediate family member.

For an expected excused absence, contact me by university email **in advance**. For an emergency, contact me as soon as you reasonably can, with documentation. Students may be dropped for excessive unexcused absences in accordance with UTRGV's attendance policy.

**Thursday, November 5** is the last day to drop with a grade of W. The midterm is returned on October 19 so that you have a real grade in hand well before that date.

## University Policy on Recordings in Class

The use of classroom recordings is governed by the Federal Educational Rights and Privacy Act (FERPA), UTRGV's acceptable-use policy, and UTRGV HOP Policy STU 02-100 Student Conduct and Discipline. A recording of class sessions will be kept and stored by UTRGV, in accordance with FERPA and UTRGV policies. Your instructor will

not share the recordings of your class activities outside of course participants, which include your fellow students, teaching assistants, or graduate assistants, and any guest faculty or community-based learning partners with whom we may engage during a class session. You may not share recordings outside of this course. As referenced in UTRGV HOP Policy STU 02-100 Student Conduct and Discipline, doing so may result in disciplinary action. This is an instruction from university officials.

## Course Use of Artificial Intelligence (AI)

AI assistants are a normal part of how software is written now, and pretending otherwise would not prepare you for the work. They are permitted in this course, with one line drawn clearly.

**Encouraged.** Use an AI assistant to explain a compiler error you do not understand, to work out what a piece of assembly is doing, to hear a concept from lecture put in different words, to find the right technical vocabulary for a written answer, or to review code you have already written and tell you what is wrong with it.

**Not permitted.** Submitting code as your own work when you cannot explain how it works, line by line, on request. This applies whether the code came from an AI assistant, a classmate, a previous semester, or a web search. The source is not the issue — not understanding what you submitted is the issue.

**How this is checked.** I may ask you to walk me through any code you submit. If you can explain it, you are fine regardless of what helped you write it. If you cannot, that is an academic integrity matter under the policy below. Exams are closed-book and closed-device, so material you did not actually learn shows up there in any case.

The practical version: an assistant that explains things to you makes you faster. An assistant that writes things for you makes the midterm very hard.

## University Academic Integrity Statement

Academic integrity is fundamental in our actions, as any act of dishonesty conflicts as much with academic achievement as with the values of honesty and integrity. Violations of academic integrity constitute academic dishonesty and include, but are not limited to: cheating, plagiarism (including self-plagiarism), and collusion; submission for credit of any work or materials that are attributable in whole or in part to another person; taking an examination for another person; any act designed to give unfair advantage to a student; or the attempt to commit such acts (UT System Board of Regents Rules and Regulations, UTRGV Handbook of Operating Procedures STU 02-100, and UTRGV Academic Integrity Guidelines, Academic Dishonesty Guidelines). All violations of Academic Dishonesty will be reported to Student Rights and Responsibilities in accordance with HOP STU 02-100, Student Code of Conduct.

## Student Support Resources

| Center                         | E-mail                     |
|--------------------------------|----------------------------|
| Advising Center                | AcademicAdvising@utrgv.edu |
| Career Center                  | CareerCenter@utrgv.edu     |
| Counseling Center              | Counseling@utrgv.edu       |
| Food Pantry                    | FoodPantry@utrgv.edu       |
| Learning Center                | LearningCenter@utrgv.edu   |
| University Library             | circulation@utrgv.edu      |
| Writing Center                 | WC@utrgv.edu               |
| UCentral                       | ucentral@utrgv.edu         |
| Brightspace Assistance (COLTT) | Ticket Portal              |

## University Policy Statements

### Sexual Misconduct and Mandatory Reporting

In accordance with UT System and institutional policies, your instructor is a “Responsible Employee” for mandatory reporting purposes under Title IX regulations and Texas Education Code sections 51.251-252 and must report any instance of sexual misconduct by or against a student or employee of the institution, which includes sexual assault,

stalking, dating violence, domestic violence, and sexual harassment to the Office of Title IX and Equal Opportunity (otixeo@utrgv.edu). More information can be found on the OTIXEO website. If students, faculty, or staff would like confidential assistance, or have questions, they can contact OAVP (Office for Advocacy & Violence Prevention).

### **Student Accessibility Services**

Student Accessibility Services has offices on Brownsville and Edinburg campuses. Visit the SAS web page to learn more and explore accessibility services.

### **Students with Disabilities**

Students with a documented disability (physical, psychological, learning, or other disability which affects academic performance) who would like to receive reasonable academic accommodations should contact Student Accessibility Services (SAS) for additional information. The student must apply for accommodations using the mySAS portal and is responsible for providing sufficient documentation of the disability to SAS. Upon submission of the request, students should expect to participate in an interactive discussion, or an intake appointment, with SAS staff. Accommodations may be requested at any time but are not granted retroactively, meaning they are valid moving forward after approval by SAS. Academic accommodations will not apply prior to SAS' approval. Students should contact SAS early in the semester/module for guidance.

Students who experience a broken bone, severe injury, or undergo surgery may also be eligible for temporary accommodations. Please contact Student Accessibility Services (SAS) for more information.

### **Pregnancy, Pregnancy-Related, and Parenting Accommodations**

Title IX of the Education Amendments of 1972 prohibits discrimination based on sex, which includes discrimination based on pregnancy, marital status, or parental status.

Students seeking accommodations related to pregnancy, pregnancy-related condition, or parenting should submit the request using the form found at Pregnancy and Parenting | UTRGV.

### **Mandatory Course Evaluation Period**

Students have the opportunity to complete an ONLINE evaluation of this course through Watermark Course Evaluations and Surveys, which may be accessed through my.UTRGV or the Brightspace course module. Course evaluations are used by the instructor to inform revisions of the course to ensure student success. Course evaluations are also used by the instructor for annual performance review, promotion applications, teaching award applications, among others.

### **Acknowledgments**

This course is modeled after Harvard's CS 61: Systems Programming and Machine Organization by Eddie Kohler. Several assignments are adapted from the CS:APP lab materials by Bryant and O'Hallaron.